

6 I/O 系统

6.1 I/O 系统

概括起来讲，计算机就是加工处理数据的，只是在不同的应用领域中可能面临不同的问题，需要加工处理的数据在类型、结构、规模、方式以及来源、去向上可能有些差别。

利用计算机系统加工处理数据需要解决的基本问题体现在四个方面：输入数据、存储数据、处理数据、输出数据。

一般来说，需要利用计算机系统加工处理的数据首先需要输入到系统内部，在系统内部按需加工处理完毕后，还需要进行相应的输出，比如存放到外存储器，或者打印出来，或者通过网络传输到其它系统，或者送给一些控制设备等。

为此，计算机系统需要配备相应的输入、输出设备（I/O 设备，统称外围设备），来实现数据的输入、输出。而且由于数据来源和去向的多样化，输入、输出设备也呈现多样化的趋势。

比如，由于技术的限制，早期的计算机大多只配备标准的终端设备，包括一个键盘和一个显示器。后来，随着应用的需要和技术的发展，新的 I/O 设备不断出现，比如：打印机、鼠标、硬盘、绘图仪、扫描仪、触摸屏、光笔等。随着计算机网络的发展，数据还可以通过网络传输；随着人工智能技术的发展，还出现了语音识别设备、图像识别设备等。

可以肯定的是，随着新的应用领域的出现和新技术的发展，新的 I/O 设备会不断出现。

一般来说，硬件离不开软件的驱动，软件控制硬件按我们的意图做

我们想做的事，I/O 设备也是如此。

计算机系统中分布在主机周围的各种 I/O 设备及其相关电路和控制软件构成了计算机系统的 **I/O 系统**，其作用就是实现数据的输入、输出。

6.2 I/O 接口

I/O 设备需要和主机进行有效连接，才能实现数据的输入、输出。由于各方面的原因，I/O 设备通常又不能直接和主机相连，解决办法就是在 I/O 设备和主机双方之间增设一个中间电路，双方通过增设的中间电路交换数据，这个中间电路（中间装置）就是 **I/O 接口**。

6.2.1 I/O 接口存在的原因

I/O 接口存在的原因是多方面的，这和 I/O 设备的多样化有关。不同的 I/O 设备在输入、输出的数据格式、速度以及输入、输出数据的具体方式等方面存在一定的差异。

有些 I/O 设备输入、输出的是串行数据，而主机处理的是并行数据；有些 I/O 设备输入、输出的也是并行数据，但是在细节方面可能存在差异；有些 I/O 设备输入、输出的是模拟信号，需要把模拟信号转换成数字信号才能和主机交换；即使是数字信号，也可能存在电平规范的差别。

此外，为了实现更灵活的输入、输出，需要对 I/O 设备进行相应的管理与控制，可以通过 I/O 接口来管理、控制 I/O 设备，而且一个 I/O 接口还可以同时管理多个 I/O 设备。

6.2.2 I/O 系统典型结构

计算机 I/O 系统典型结构如图 1 所示。

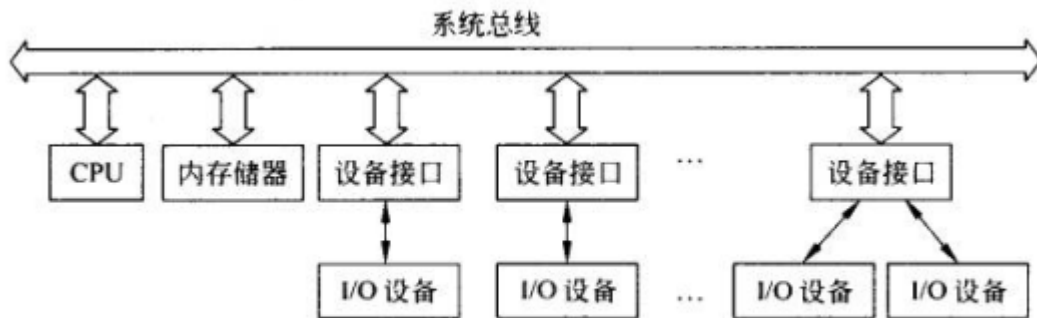


图 1 典型的输入输出系统

虽然**系统总线**作为公共信息通路，通常起到连接处理机、主存储器和外围设备的作用，但实际上外围设备并不能直接连接到系统总线上，需要通过扩展总线以及 I/O 接口来实现 I/O 设备与主机两者之间的连接。

6.2.3 I/O 接口的基本结构

为了完成 I/O 接口的功能，大多数 I/O 接口电路内部设置有若干寄存器。从作用来看，这些寄存器可以分为 3 类：数据寄存器、状态寄存器、控制寄存器。

数据寄存器：供双方交换数据使用，CPU 可以读或写。输入数据时，I/O 接口接收输入设备送过来的数据，存入数据寄存器后，供 CPU 读取；输出数据时，CPU 写入数据，供 I/O 接口发送给输出设备。

状态寄存器：记录 I/O 接口及其控制的 I/O 设备的工作状态，供 CPU 读取。CPU 通过读状态寄存器可以了解 I/O 接口及其控制的 I/O 设备的工作状态，从而决定接下来该做什么。

控制寄存器：接收 CPU 送过来的控制命令，按命令要求控制 I/O 接口及其控制的 I/O 设备的工作。

可以通过编程对这些寄存器进行读写，通过读写这些寄存器来实现主机和 I/O 设备之间的数据交换。相应地，也称这些 I/O 接口为可编程 I/O 接口。在现代计算机系统中，较复杂的 I/O 设备都配有相应的可编程 I/O 接口。

I/O 接口的典型结构如图 2 所示。

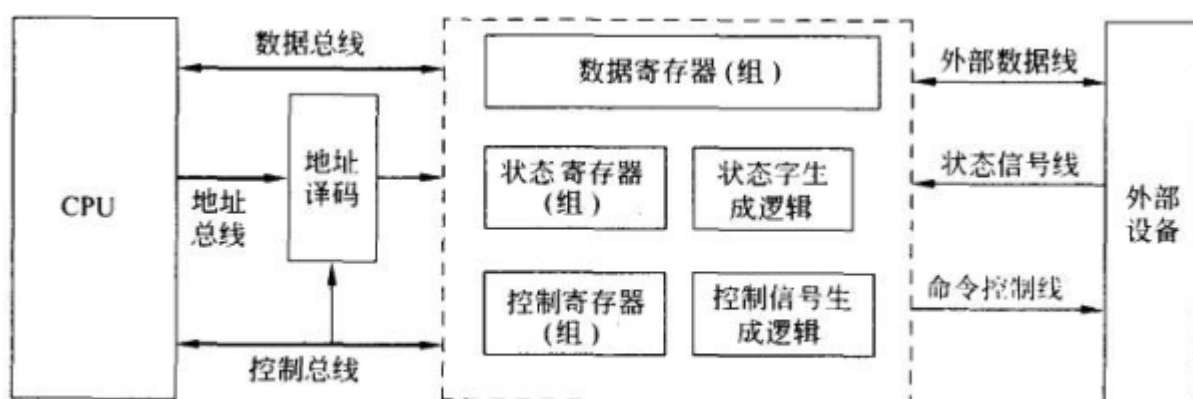


图 2 I/O 接口的典型结构

为了读写 I/O 接口内部的寄存器，I/O 接口需要具备对这些寄存器的寻址能力。

6.2.4 I/O 接口的基本功能

综上所述，I/O 接口的基本功能包括：数据格式转换、信号电平转换、并行/串行数字转换、数字/模拟信号转换、端口寻址、数据缓冲以及各种管理、控制功能等。

6.2.5 I/O 接口的分类

I/O 设备的多样性必然导致 I/O 接口的多样性。

1 并行接口和串行接口

并行接口用来并行传送数据，也就是将构成同一个数据的多个比特同时传送出去。按这种方式传送的数据也叫并行数据。

串行接口用来串行传送数据，也就是将构成同一个数据的多个比特一个一个地传送出去。按这种方式传送的数据也叫串行数据。

主机处理的都是并行数据，但是外围设备处理的有些是并行数据，有些则是串行数据，根据需要而定。

2 同步接口和异步接口

主机和外设交换数据需要遵守一定的规则，只有严格遵守这些规则，才能确保双方正确地交换数据。为正确地交换数据，双方共同遵守一定的规则，互相协调、配合，这就是**同步**。换言之，同步就是双方协调、配合的一种机制。

同步接口与总线的数据传送一般由统一的时钟信号来同步。这种接口的控制逻辑较为简单，但要求 I/O 设备与 CPU、主存在速度上必须能够很好的匹配，这在某种程度上限制了所使用 I/O 设备的种类与型号。在实际应用中，考虑到系统的灵活性，一般允许 I/O 操作总线周期的时钟脉冲个数可以在一定范围内变化，即总线时段的长短可以灵活划分。

异步接口与系统总线之间一般采用请求-应答方式进行同步。通常把处理机作为主设备，而某一 I/O 设备作为从设备。主设备首先提出要求交换信息的请求信号，经总线和接口传递到从设备，从设备完成主设备指定的操作后，又通过接口和总线向主设备发出应答信号。整个信息交换过程总是这样请求-应答、请求-应答地进行着，从请求到应答的间隔时间是由操作的实际时间决定的。

3 数字接口和模拟接口

有些 I/O 设备或者装置可以直接发送/接收数字量；而有些 I/O 设备或装置只能发送/接收模拟信号，尤其是在控制领域。

数字接口发送/接收并处理数字量（数字信号）的接口，模拟接口发送/接收并处理模拟量（模拟信号）的接口。

由于主机处理的总是数字量，因此必要时需要在数字量和模拟量之间进行转换，于是就有了数字量和模拟量之间的转换接口。

除了以上分类外，还可以根据接口传送数据的方式等对接口进行分类。

6.3 端口

CPU 通过 I/O 接口和 I/O 设备打交道，通过控制 I/O 接口来间接控制 I/O 设备。CPU 控制 I/O 接口的方式就是对 I/O 接口里面的寄存器进行相应的读写操作。

6.3.1 端口及其编址方式

I/O 接口内部的寄存器也称为**端口**，有别于 CPU 内部的寄存器。在主机周围，分布在不同 I/O 接口内部的端口合计起来还不少。

CPU 访问这些端口时，最基本的区分方式就是利用**端口地址**，就像内存地址一样。

端口地址的编排方式有两种：统一编址和独立编址。

1 统一编址

统一编址将内存单元和端口合并起来，统一编排地址。这种方式相当于将一个端口视同一个内存单元，所以也称为存储映象方式。

统一编址的特点是内存和端口共同瓜分同一地址空间，其中一部

分为内存所有，另一部分为端口所有；优点是同一个地址，要么是内存地址，要么是端口地址，可以用统一的指令进行访问；地址译码时，不用区分是内存地址还是端口地址；缺点是地址也是一种资源，在地址空间大小固定（资源有限）的情况下，端口占用一部分，内存就只能使用剩余的部分。

2 独立编址

独立编址内存单元和端口分开编排地址，互相独立。独立编址的特点是内存和端口分别使用不同的地址空间，优点是内存能使用的地址资源更多，缺点是同一个地址，可能是内存地址，也可能是端口地址，必须用不同的指令进行访问；地址译码时，需要区分是内存地址还是端口地址。

两种编址方案如图 3 所示。

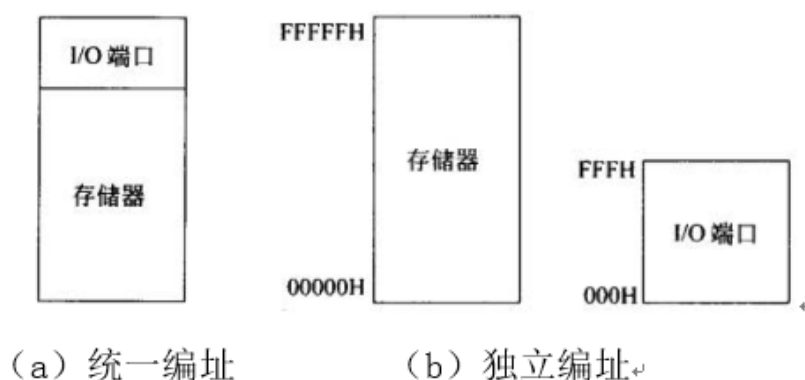


图 3 两种编址方式对比

6.3.2 端口地址译码

就像对内存单元的访问一样，主机在访问端口时，接口需要根据端口地址先经过译码后选中对应的端口，然后在读/写控制信号的作用对选中的端口进行读/写。

由于端口分布在不同的接口电路中，访问端口时需要先找到目标端口所在的接口电路，即目标接口；然后在目标接口内部，再进一步找到目标端口。

因此端口地址译码由片间的地址译码和片内的地址译码两部分构成。片间的地址译码也叫**片选**，就是指根据所给的端口地址等信息选中目标接口电路；片内的地址译码也叫**字选**，就是指根据所给的端口地址等信息在目标接口电路内部进一步选中目标端口。

实现字选的片内地址译码问题通常由设计接口电路时所选用的接口芯片内部解决，因此设计接口电路时着重关注的是片间的地址译码问题，即片选。

实现片选的端口地址译码电路的设计方法和实现片选的内存地址译码电路的设计方法基本一致，可以利用**译码电路**进行译码选中相应的接口，也可以直接采用**线选法**选中相应的接口。

利用译码电路实现片选的示意图如图 4 所示，其中的译码电路通常就是一个组合逻辑电路。

所谓线选法就是直接利用专门的信号线来对接口芯片进行选择，一根线控制一个接口芯片，控制线有效时选中相应的芯片，无效时不选中，如图 5 所示。

采用译码电路进行译码时，可以选择专门的**集成译码器**辅助门电路，也可以选择**可编程逻辑器件**，来设计出所需的译码电路。

参与地址译码的系统地址信号可以是**全部地址**，也可以是**部分地址**。

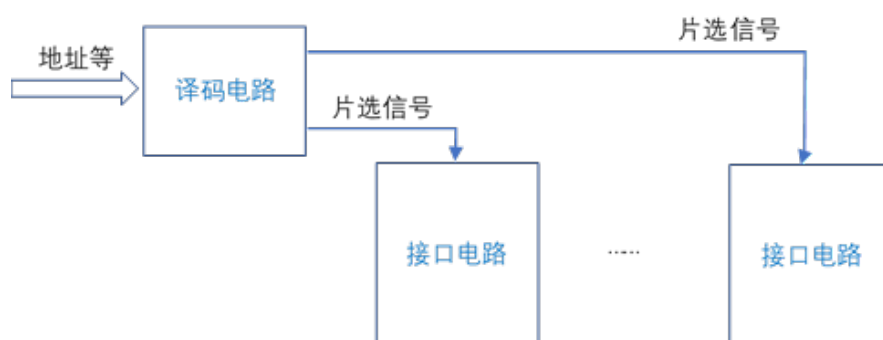


图 4 译码电路译码

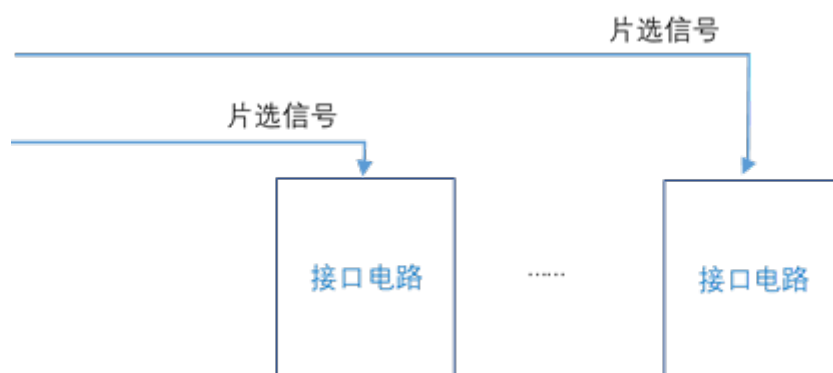


图 5 线选法

6.4 输入输出方式

为满足不同应用的需要，主机和 I/O 设备之间的数据传送方式可以分为：无条件传送方式、查询方式、中断方式和 DMA 方式等。

6.4.1 无条件传送方式

无条件传送方式是指双发发送/接收数据是无条件的，任何时候想发就发，想收就收。

无条件传送方式的数据发送/接收流程如图 6 (a) 所示。无条件传送一般适用于双方速度几乎一致，而且 I/O 设备随时处于就绪状态的场合，比如电子显示屏等。

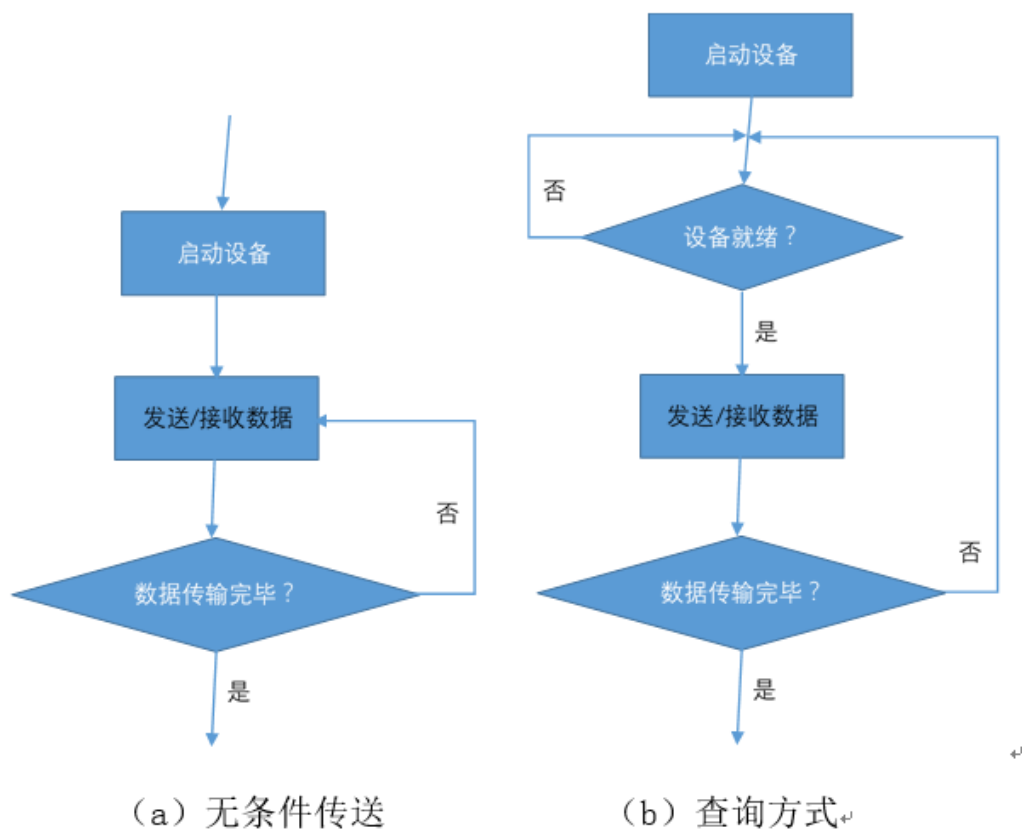


图 6 无条件传送方式和查询方式对比

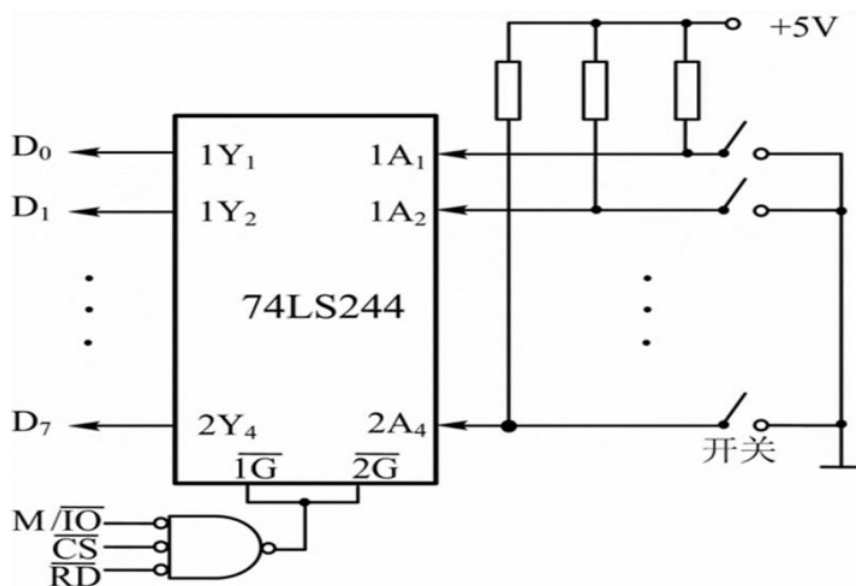


图 7 无条件输入接口电路

图 7 所示是一个比较简单的无条件输入接口电路，用来对开关状态进行采样，其中：74LS244 是缓冲器，同时也是用于输入的数据口，

其输入直接来自开关状态。

任何时候，主机需要采样开关状态时，可以直接读缓冲器。

图 8 所示是一个比较简单的无条件输出接口，用来控制 LED 灯的亮和灭，其中：74LS273 是锁存器，同时也是用于输出的数据口，其输出直接控制 LED 灯。

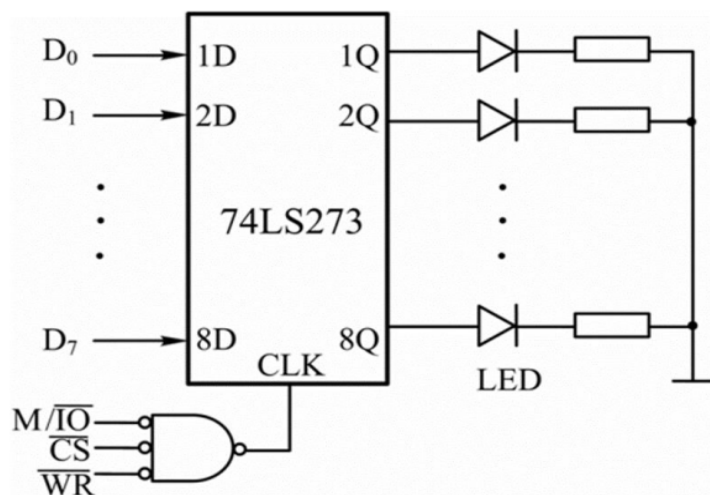


图 8 无条件输出接口电路

任何时候，主机需要控制 LED 灯时，可以直接写锁存器。

6.4.2 查询方式

很多 I/O 设备的工作速度和主机相比，都有明显的差异，比如打印机等。打印机打印一个字符的过程涉及机械运动，其打印速度显然比不上主机传送数据的速度。

在这种情况下就必须采用查询方式，也就是有条件传送方式。

有条件传送方式就是指双方发送/接收数据是有条件的，在发送/接收数据之前必须检测是否满足发送/接收数据的条件，只有具备发送/接收数据的条件才能发送/接收数据。

为此，接口电路在其内部配备有专门的状态位，用于记录是否具备

发送/接收数据的条件。同时，接口电路还设置有专门的联络控制信号，用于通知 I/O 设备是否可以发送/接收数据。

对主机来说，每次检测发送/接收数据的条件就是通过读取接口内部的状态位（位于状态端口内）并且对其进行分析实现的，因此这种方式也叫查询方式。

查询方式的数据发送/接收流程如图 6（b）所示。

图 9 所示是一个简单的查询式输入接口电路，其中：READY 是输入设备用来通知主机输入数据已经准备好的联络信号，选通信号是输入设备用来将数据锁存进锁存器的控制信号。

对主机来说，READY 为 1，说明有数据可读；否则，无数据可读。主机每次输入数据前都得检测 READY。

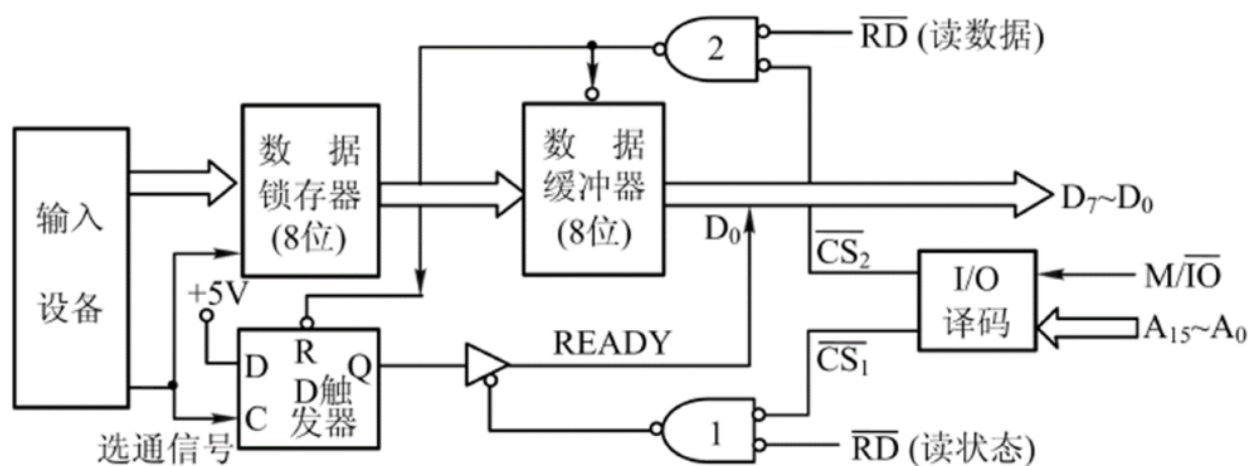


图 9 一个简单的查询式输入接口

需要输入数据时，输入设备准备好数据后利用选通信号将数据锁存到数据锁存器，同时使 D 触发器的状态 Q 置 1（建立 READY）。

主机一侧则执行以下操作：

Step1: 读状态口（ $\overline{CS}_1$ 和 $\overline{RD}$ 有效），将触发器的状态 Q 作为 READY

信号经数据线 (D_0) 读入

Step2: 检测 D_0 , 如果 D_0 为 1, 说明有数据可读, 随后读数据口 ($\overline{CS}_2$ 和 $\overline{RD}$ 有效), 将数据读入; 同时触发器清零 (撤销 READY)。如果需要继续输入数据, 回到 Step1。如果 D_0 为 0, 说明无数据可读, 回到 Step1。

以上过程会反复进行, 直到数据输入完毕。

图 10 所示是一个简单的查询式输出接口电路, 其中: BUSY 是输出设备用来通知主机其正在处理输出数据的联络信号, $\overline{ACK}$ 是输出设备用来撤销 BUSY 的控制信号。

对主机来说, BUSY 为 1, 说明输出设备正忙着处理上一次的数据, 不能再发送数据; 否则, 说明输出设备已经处理完上一次的数据, 可以发送下一个数据。

对输出设备来说, BUSY 为 1, 说明有数据需要处理; 否则没有需要处理的数据。

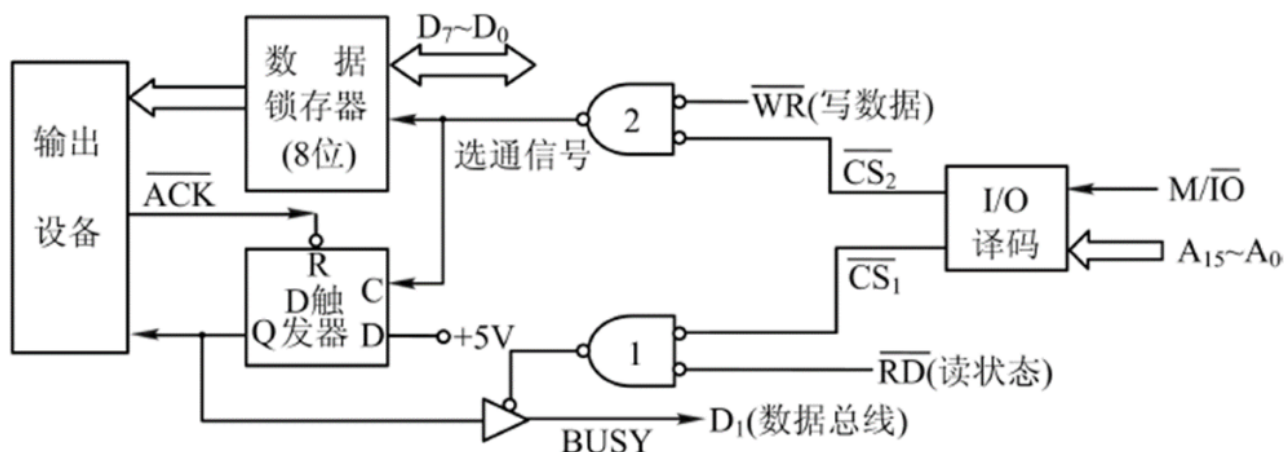


图 10 一个简单的查询式输出接口

需要输出数据时, 输出设备检测触发器的状态: 如果其状态为 1, 说明有数据需要处理, 处理完数据后利用 $\overline{ACK}$ 复位触发器(撤销 BUSY)。

主机一侧则执行以下操作：

Step1: 读状态口 ($\overline{CS}_1$ 和 $\overline{RD}$ 有效)，将触发器的状态 Q 作为 BUSY 信号经数据线 (D_1) 读入。

Step2: 检测 D_1 ，如果 D_1 为 0，说明可以发送下一个数据，随后写数据口 ($\overline{CS}_2$ 和 $\overline{WR}$ 有效)，将数据写入数据锁存器；同时触发器置位 (建立 BUSY)，通知输出设备处理数据。如果需要继续输出数据，回到 Step1。如果 D_1 为 1，说明不能发送下一个数据，回到 Step1。

以上过程会反复进行，直到数据输出完毕。

在以上例子中，选通信号、READY 信号以及 $\overline{ACK}$ 信号、BUSY 信号是双方为保证正确传输数据用来进行控制联络的专门信号。

一般来说，双方通过接口交换数据时，需要做好同步工作，基本规则是：先发数据，再收数据，再发数据，再收数据，……。

为此，发送方在发送数据前需要知道接收方是否已经准备好接收数据，而接收方需要知道是否有数据需要接收。

确保双方按此规则发送/接收数据的基本手段就是设置专门的控制联络信号供双方使用，比如上述的选通信号、READY 信号以及 $\overline{ACK}$ 信号、BUSY 信号。

双方利用控制信号建立联络信号也叫**握手**，控制联络信号也叫**握手信号**。握手需要遵守的规则也叫**握手协议**，换言之，握手协议就是关于“谁先做什么，然后谁再做什么；如果怎么样，就怎么样；否则又该怎么样；……”的一套规则。双方必须严格遵守握手协议，否则可能导致错误。

6.4.3 中断方式

前述无条件传送方式和查询方式的共同特点是：主机通过执行专门的数据传送程序来完成数据的传送，且在数据传送期间不能做别的事情，也叫程序传送方式。

这种方式的优点是实现起来相对简单，但是缺点也很明显：主机发送/接收数据之前都要查询条件，如果条件不满足，就得继续查询，直到条件满足才能发送/接收一个数据。为发送/接收下一个数据，还得先查询条件。这样一来，查询条件会耗费大量的 CPU 时间。

1 中断

一种改进的措施是，在主机发送/接收数据的条件不满足之前，CPU 可以去做别的事情（执行别的程序），等到主机发送/接收数据的条件满足之后，I/O 设备（通过接口）主动通知 CPU，然后 CPU 暂停正在执行的程序转而执行发送/接收数据的程序，完成数据的发送/接收。等数据发送/接收完后，CPU 再恢复执行原来暂停的程序。此时的数据发送/接收程序中不再需要用于查询条件的代码段。

这种数据传送方式就是**中断方式**。所谓**中断**，就是指由于系统内外发生了什么事，CPU 暂停正在执行的程序转而去处理所发生的事，等处理完后，恢复执行原来暂停的程序。换言之，原来正常执行的程序被发生的事情暂时中断了。

中断是 I/O 设备和主机交互的一种很重要的方式。

与中断相联系的另一个重要概念是**异常**。**异常**是指异常事件引起的中断，比如：打印机缺墨水引起的异常，溢出引起的异常等。广义的

中断包括了异常，而狭义（严格意义）的中断则是指正常事件引起的中断，比如：敲击键盘引起的键盘中断，移动鼠标引起的鼠标中断等。

2 中断的基本问题

中断要解决的基本问题包括：中断请求、中断检测、中断响应、中断服务、中断返回。

中断请求：以中断方式和主机交互的 I/O 设备在适当的时候，比如输入设备准备好数据后，会通过中断请求信号线向 CPU 发送中断请求信号，通知 CPU 来取数据。

中断检测：CPU 在正常执行程序的过程中会定期对中断请求信号线进行检测。

中断响应：CPU 如果检测到中断请求信号，在系统允许中断的情况下，会通过中断响应信号线向发出中断请求的 I/O 设备发送中断响应信号，通知 I/O 设备其中断请求得到响应，并暂停正在执行的程序，转而进入中断服务程序。

中断服务：执行中断服务程序，处理所发生的事，比如发送/接收数据。

中断返回：结束执行中断服务程序，转而回到原来暂停执行的程序。上述过程也就是系统处理中断的一般过程。

中断服务程序：事先编写好的一段程序，CPU 通过执行中断服务程序来处理所发生的事，比如发送/接收数据。

中断源：系统中能够引起中断的各种原因（事件），即中断的来源。

单中断系统如图 11 所示，其中：INTR 和 INTA 分别为中断请求信

号线和中断响应信号线。

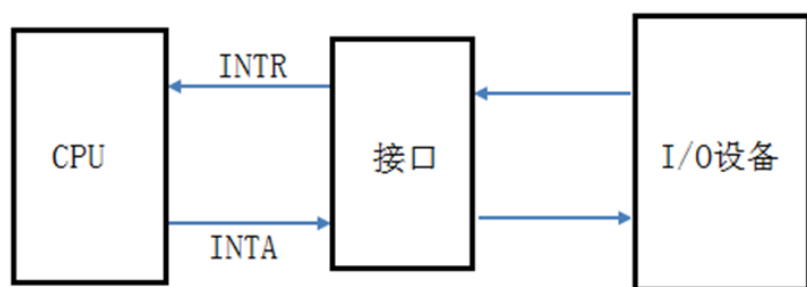


图 11 单中断源系统

单中断处理的一般过程如图 12 所示。

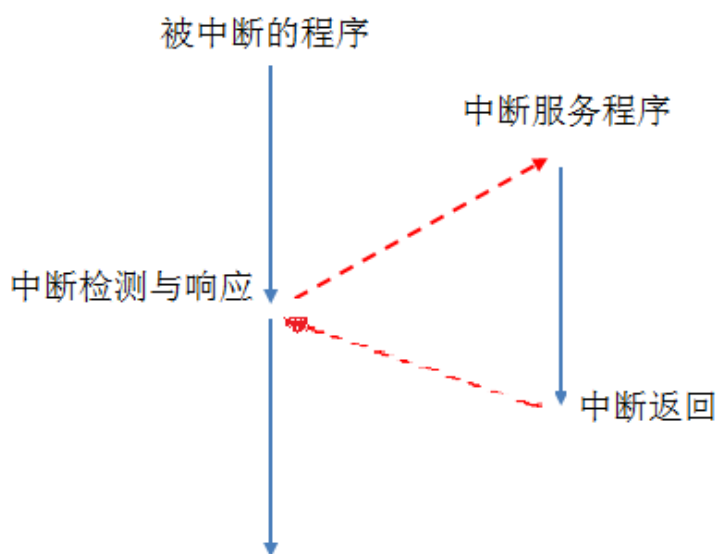


图 12 单中断处理过程

3 多中断系统

由于系统中通常有多个 I/O 设备，所以通常系统中同时有多个中断源存在，在一段时间内就可能同时产生多个中断请求。

同时有多个中断请求时，该优先处理谁呢？

解决的办法就是给不同的中断源赋予不同的优先级。当不同优先级的中断源同时产生中断请求时，优先处理级别最高的中断请求。

中断优先级：通常用无符号数来区分不同的中断源的优先级别。

在处理当前级别最高的中断请求期间，如果来了级别更高的新的

中断请求，又该如何处理呢？

在不允许中断处理又被中断的情况下，新来的级别更高的中断请求只能等待 CPU 处理完目前正在处理的中断，从正在执行的中断服务程序返回后，再处理新来的更高级别的中断。

如果允许中断处理又被中断，正在处理的中断就会被新来的更高级别的中断中断，从而形成嵌套。

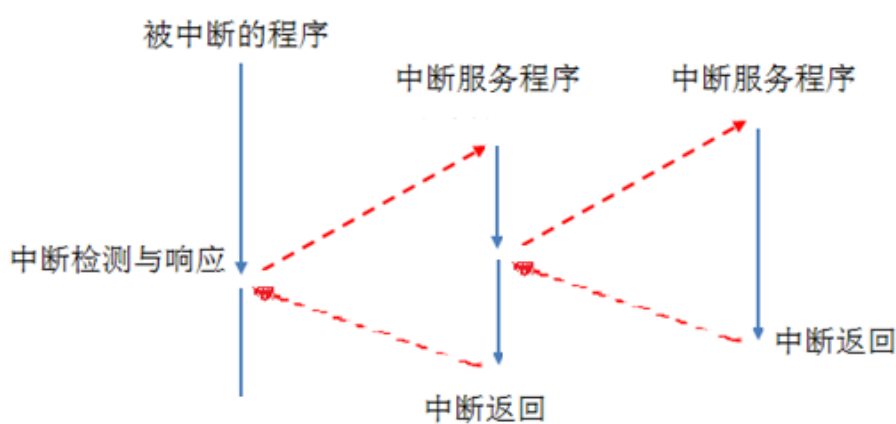


图 13 中断嵌套

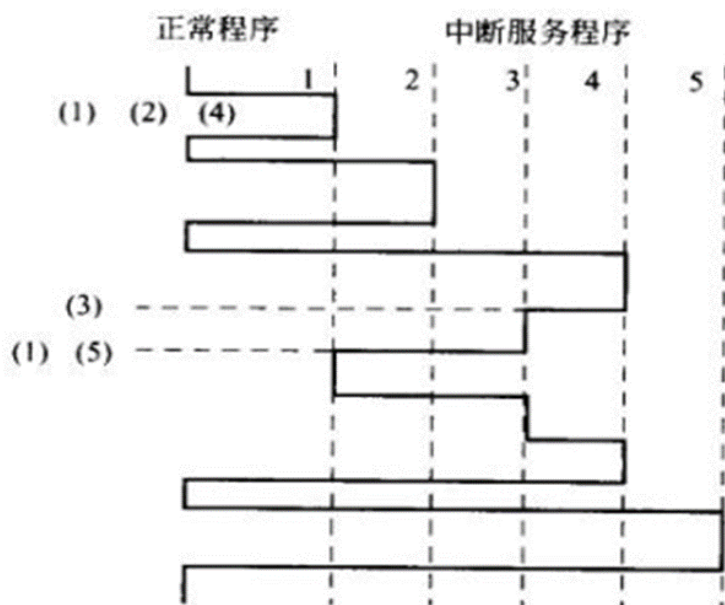


图 14 中断处理过程

中断嵌套：在执行某个中断服务程序的过程中，由于某种原因，需

要暂停正在执行的中断服务程序，转而去处理新的中断，等新的中断处理完毕后，又恢复执行被暂停的中断。这一点就像子程序或函数嵌套调用一样，如图 13 所示。

多中断处理过程如图 14 所示。

4 中断管理器

各个中断源没必要都直接向 CPU 发送请求，可以在系统中设置一个专门的部件，负责集中管理这些中断，接收并记录它们的请求，并根据一定的规则将它们的请求转达给 CPU，由 CPU 进行适时处理。

中断管理器：系统中专门负责集中管理各个中断源的部件。

早期 PC 上经常使用的一种中断管理器如图 15 所示。

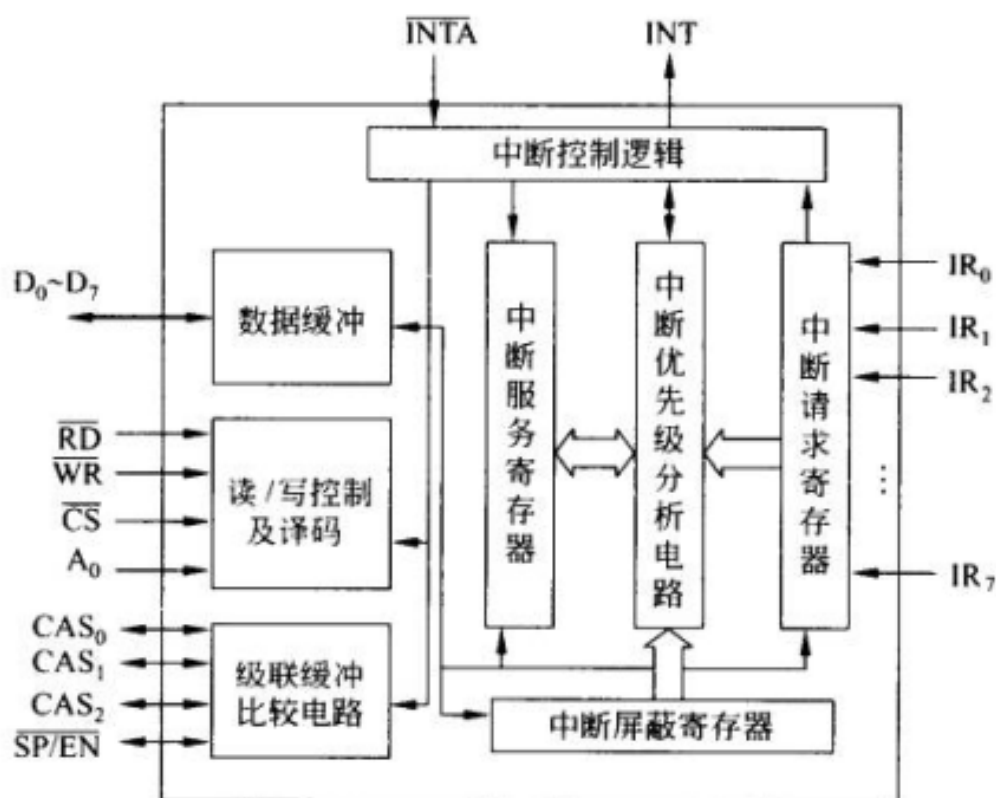


图 15 Intel 8259 中断管理器

其中：中断请求寄存器用于接收并记录各个中断源的中断请求；中

断屏蔽寄存器用于对各个中断源的中断请求进行屏蔽管理，被屏蔽的中断请求不会被处理；**中断服务寄存器**用于记录目前正在处理以及被再次中断而尚未处理完毕的中断请求；**中断优先级分析电路**用于对未被屏蔽的中断请求进行优先级比较分析。

中断管理器和 CPU 双方协作共同处理中断的过程大致如下：

- 1) 各个中断源向中断管理器发中断请求；
- 2) 中断管理器记录大家的请求
- 3) 如果 CPU 目前没有处理任何中断，中断管理器向 CPU 发中断请求；如果 CPU 目前正在处理某个中断且新来的中断中有比正在处理的中断级别更高的中断，中断管理器也向 CPU 发中断请求
- 4) 在允许中断（包括中断正在处理的中断）的情况下，CPU 适时检测中断管理器发送的中断请求
- 5) CPU 检测到中断管理器发送的中断请求后，向中断管理器发送响应信号并开始中断响应过程：暂停正在执行的程序或者中断服务程序，保护相关的重要信息。
- 6) 中断管理器接收到 CPU 发送的中断响应信号后，向 CPU 提供级别最高或者新来的级别最高的中断请求的相关信息
- 7) CPU 利用中断管理器提供的级别最高的中断请求的有关信息找到其中断服务程序的起始位置
- 8) 开始执行中断服务程序
- 9) 在中断服务程序的最后通过执行专门的中断返回指令回到被暂停的程序或中断服务程序

5 向量中断和查询中断

系统处理中断的具体过程与中断系统的设计与实现方案有关，常见的基本方案有两种：向量中断和查询中断。

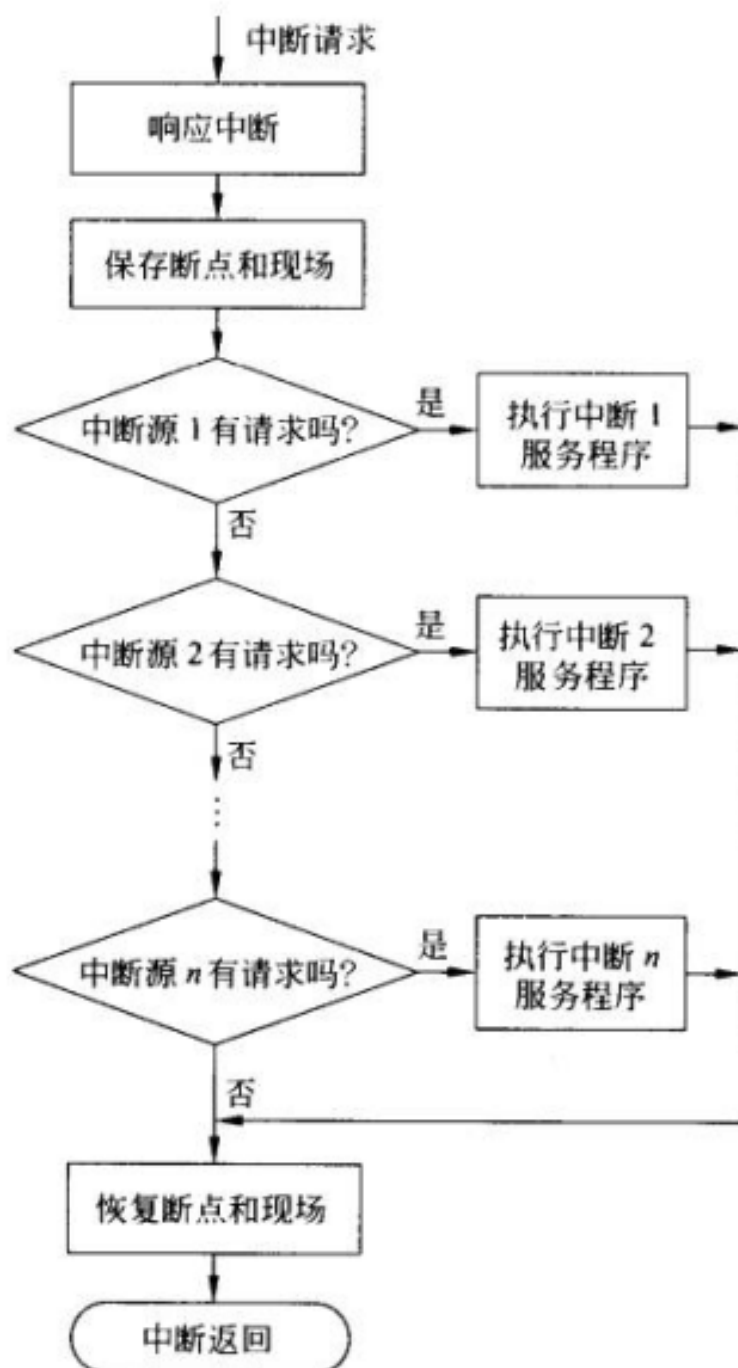


图 16 查询中断

中断类型码：用来区分不同中断源及其中断服务程序的无符号数。

中断向量：中断服务程序在内存中的起始位置，也就是入口地址。

中断向量表：严格按中断类型码由小到大的顺序集中存放所有中断向量的地址表，通常位于内存某固定区域。

向量中断：中断管理器在收到 CPU 的响应信号后，适时向 CPU 提供级别最高或者新来的级别最高的中断请求的中断类型码。CPU 在响应期间收到此中断类型码后据此查询中断向量表，从中找到所需的中断向量，然后利用找到的中断向量转入相应的中断服务程序。

在没有中断管理器集中管理中断源的情况下，可以采用**查询中断**。这种方式的特点是在 CPU 检测到有中断请求产生时，在中断响应期间通过执行查询程序确定其中级别最高的中断源，并转入响应的中断服务程序。

查询中断的处理过程如图 16 所示，查询顺序决定了优先级。

查询中断的优点是实现相对简单，缺陷是中断响应期间执行查询程序确定级别最高的中断源同样需要消耗 CPU 时间，这一点本身就违背了引入中断机制的出发点。

必要时，向量中断和查询中断可以结合使用。

6.4.4 DMA 方式

中断方式虽然不需要 CPU 查询发送/接收数据的条件，但是最终的数据传送还是要靠 CPU 执行相应的程序（中断服务程序）来完成。

1 DMA 方式

为进一步将 CPU 从数据 I/O 中解放出来，尤其是需要在内存和外设之间传送数据时，可以考虑在内存和外设之间增设一个专门部件，由

它来代替 CPU 负责在内存和外设之间传送数据，这种数据传送方式就是 **DMA 方式**。

DMA 控制器：代替 CPU 负责在内存和外设之间传送数据的专门部件。

计算机系统软件（包括程序和数据）平时都存储在外存上。在程序执行过程中，需要用到时再将它们复制到内存。

指令及数据的局部性：程序在执行过程中，在一段时间内要执行的指令或要访问的数据往往集中在一起。

由于上述局部性，需要用到的程序和数据没必要一次性全部复制到内存，只需要将一段时间需要用到的部分在其第一次需要用到时复制到内存即可。

这样做有诸多好处，比如：可以利用有限的内存空间执行大规模程序，处理大规模数据；可以在不同任务间共享内存空间；等等。

当然，这样做也有其缺陷，就是需要在内存和外存之间频繁交换数据块。如果这种数据交换直接由 CPU 负责控制完成，势必消耗大量 CPU 时间。解决的办法就是引入 DMA 机制，每当需要在内存和外设之间交换数据时，由 DMA 控制器来代替 CPU 负责完成数据交换。

2 DMA 的基本问题

DMA 方式需要解决的基本问题与中断方式类似。

DMA 方式要解决的基本问题包括：DMA 请求、DMA 检测、DMA 响应、DMA 传送、DMA 结束。

DMA 请求：DMA 控制器负责接收外设的 DMA 请求，并及时向 CPU 提

出请求。

DMA 检测：CPU 需要定期检测 DMA 控制器发出的请求

DMA 响应：CPU 检测到 DMA 控制器发出的请求后，需要对 DMA 控制器做出应答并让出总线的控制权

DMA 传送：DMA 控制器接管总线控制权，并利用总线传输数据

DMA 结束：数据传输结束后，DMA 控制器归还总线给 CPU，并通知 CPU 数据传送结束。

系统处理 DMA 请求的过程与处理中断的过程也基本类似，主要差别体现在：1) 中断是通过 CPU 执行程序完成数据传输（是一种软件办法），而 DMA 是利用 DMA 控制器完成数据传输（是一种硬件办法）；2) 中断一般允许嵌套，而 DMA 一般不允许嵌套。

当然，中断管理器和 DMA 控制器都需要事先通过编程进行设置，而且 DMA 结束后通常还需要利用中断做随后该做的事情。

3 DMA 控制器与 CPU 共享总线的方式

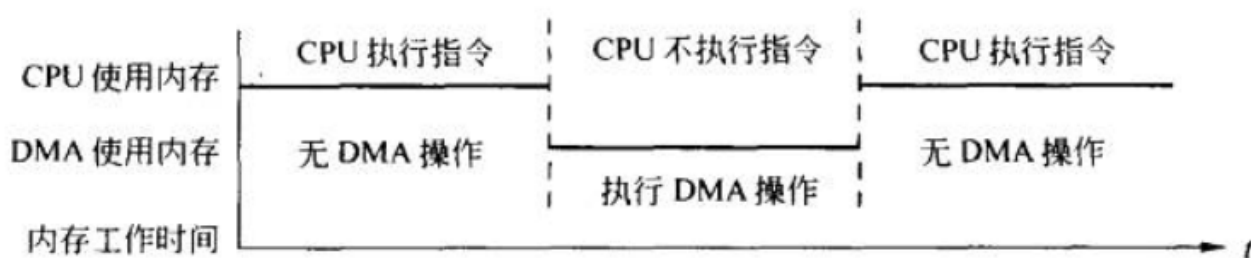
通常 DMA 控制器和 CPU 共享系统总线，在 DMA 控制器控制传输数据时，CPU 必须放弃对系统总线的控制，而由 DMA 控制器来控制系统总线。不同的计算机系统会采用不同的方法来解决 CPU 与 DMA 控制器共享总线的问题。

CPU 与 DMA 控制器共享总线大致有 3 种方式，如图 17 所示。

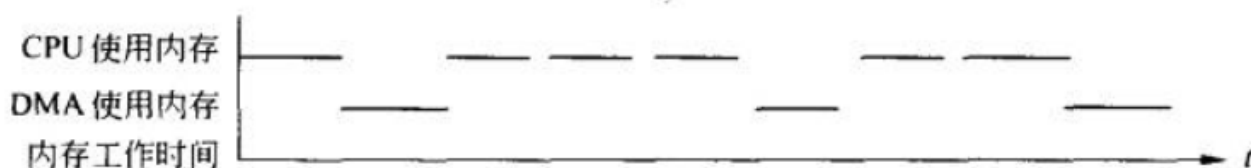
(1) CPU 暂停方式

CPU 响应 DMA 请求后，让出系统总线给 DMA 控制器使用，直到数据全部传送完毕后，DMA 控制器再把总线交还给 CPU。在此期间 CPU 是不

能访问主存的，因此 CPU 需要暂时停止工作。此时 CPU 内部的控制器要在 CPU 内部封锁时钟信号，并使 CPU 与总线之间的信号线呈现高阻状态。DMA 控制器获得总线控制权以后，开始进行数据传送。在一批数据传送完毕后，DMA 控制器通知 CPU 可以使用内存，并把总线控制权交还给 CPU，如图 17（a）所示。



(a) CPU 暂停方式



(b) 周期挪用方式



(c) 交替访问内存方式

图 17 DMA 控制器与 CPU 共享总线的方式

采用这种工作方式的 I/O 设备需要在其接口控制器中设置一定容量的存储器作为**数据缓冲存储器**使用，I/O 设备与数据缓冲存储器交换数据，主存也只与数据缓冲存储器交换数据，由于数据缓冲存储器的存取速度较快，这样可以减少由于执行 DMA 数据传送而占用系统总线的时间，从而减少了 CPU 暂停的时间。

这种控制方式比较简单，用于高速 I/O 的成批数据传送是比较合适的。缺点是 CPU 的工作会受到明显的延误，当 I/O 数据传送时间大于主存周期时，主存的利用不够充分。

数据缓冲技术：计算机系统中不同的部件（如主机和外设）之间、不同的进程之间、同一进程的不同线程之间等等，它们之间经常需要交换数据。

实现双方数据交换的最简单高效的方式就是设立数据缓冲区，一方适时往里放数据，另一方适时从中取数据。缓冲区是双方交换数据的中转站，不仅可以暂时存放数据，还可以在在一定程度上缓解双方速度不匹配带来的问题。

缓冲区的容量可以综合考虑双方速度上的差异以及数据交换方式、应用场景等灵活确定。

（2）周期挪用方式

这种方式有时也被称作周期窃取方式，如图 17（b）所示。在这种方式中，当 I/O 设备无 DMA 传送请求时，CPU 正常访问主存。当 I/O 设备需要使用总线传送数据时，产生 DMA 请求。DMA 控制器把总线请求发给 CPU，此时若 CPU 本身无使用总线的要求，CPU 就可把总线交给 DMA 控制器，由 DMA 控制器控制 I/O 设备使用总线，这样的情形当然最为理想；如果此时 CPU 也要使用总线，则 CPU 自身进入一个空闲总线周期状态，即 CPU 让出一个总线周期给 DMA 控制器（也称 DMA 控制器挪用一個总线周期）。DMA 控制器利用此总线周期控制传送一个数据字后，再把总线交还给 CPU，以便 CPU 可以执行总线操作。可见当 I/O 设备与

CPU 同时都要访问主存而出现访问主存冲突时，I/O 设备访问的优先权高于 CPU 访问的优先权，因为 I/O 设备每次占用总线的时间较短（仅一个总线周期）。

周期挪用方式能够充分提供 CPU 与 I/O 设备的利用率，是当前普遍采用的方式。其缺点是，每传送一个数据，DMA 都要产生访问请求，待到 CPU 响应后才能传送，因此判优操作及总线切换操作非常频繁，其花费的时间开销较大。往往在传输一个数据块时，需要 DMA 控制器多次申请使用总线，这影响了 DMA 的数据传输速度，这种情况适用于 I/O 设备接口控制器中数据缓冲器容量不大的场合，例如在接口控制器中仅设置一个数据寄存器的情形，对具有较大容量的数据缓冲存储器的高速外设来说是不合适的。

（3）交替访问方式

这种方式如图 17（c）所示。使用这种方式的前提是 CPU 的工作速度相对较慢，而内存的工作速度较快，或者人为拉长 CPU 执行指令的时间。如主存的存取周期为 Δt ，而 CPU 每隔 $2\Delta t$ 才产生一次访存请求，那么在 $2\Delta t$ 内，一个 Δt 供 CPU 访问主存，另一个 Δt 供 DMA 访问主存。这种方式比较好地解决了 CPU 与 I/O 设备之间的访存冲突以及设备利用不充分的问题，而且不需要有请求总线使用权的过程，总线的使用是通过分时控制的，此时 DMA 的传送对 CPU 没有影响。但加大了控制器的设计与实现难度，且对存储器的工作速度要求较高，增加了主存的成本。

4 DMA 控制器的基本结构

DMA 控制器的基本结构如图 18 所示，主要包括：寄存器组、DMA 控

制逻辑和中断控制逻辑。

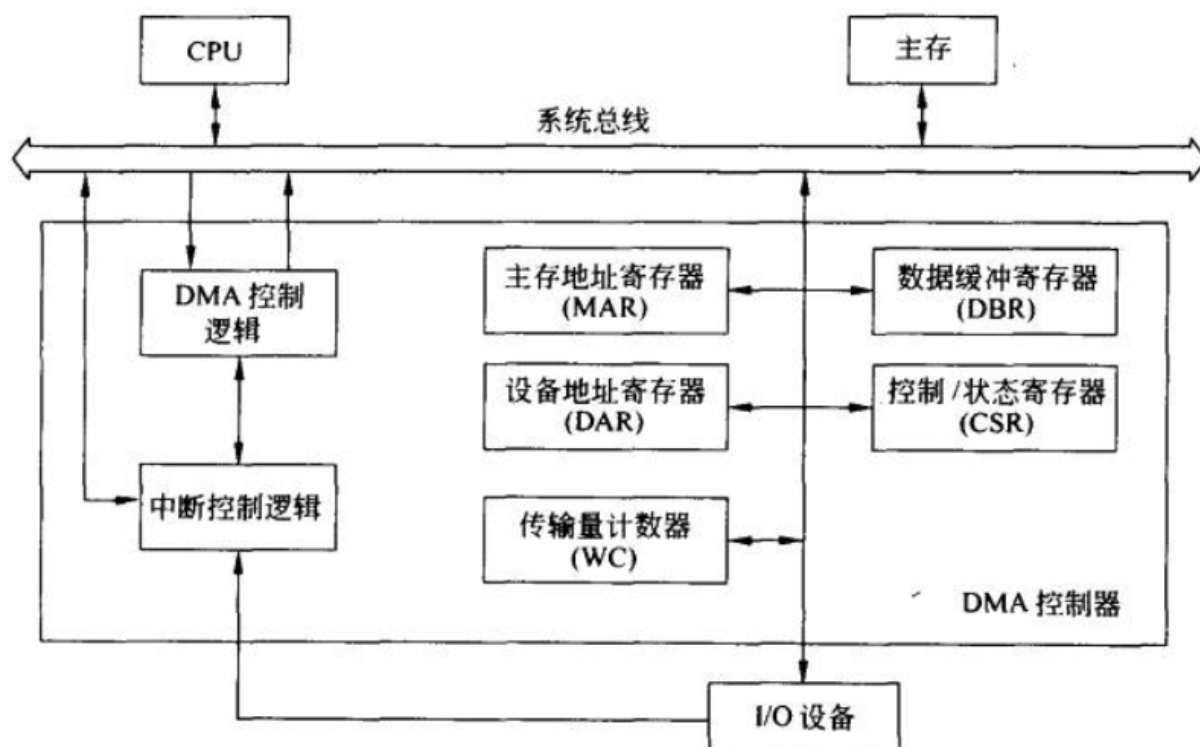


图 18 DMA 控制器的基本结构

(1) 寄存器组

寄存器组用来提供 DMA 过程中需要用到的各种参数，其中：主存地址寄存器为访问内存提供地址，每访问一个单位的数据会自动修改；设备地址寄存器为访问外围设备提供地址；传输量计数器用于控制需要传输的数据总量，传输一个单位的数据会自动修改，通常采用倒数方式；数据缓冲寄存器在需要时用来暂时存放数据。

(2) DMA 控制逻辑

DMA 控制逻辑负责完成 DMA 的预处理（初始化各类寄存器）、接收设备控制器送来的 DMA 请求信号、向设备控制器回答 DMA 允许（应答）信号、向系统申请总线以及控制总线实现 DMA 传输控制等工作。

(3) 中断控制逻辑

DMA 中断控制逻辑负责在 DMA 操作完成后向 CPU 发出中断请求，申请 CPU 对 DMA 操作进行后处理或进行下一次 DMA 传送的预处理。

5 DMA 数据传输过程

DMA 控制器控制下的数据传送过程如图 19 所示，可分为三个阶段：DMA 传送前预处理阶段、数据传送阶段及传送后处理阶段。

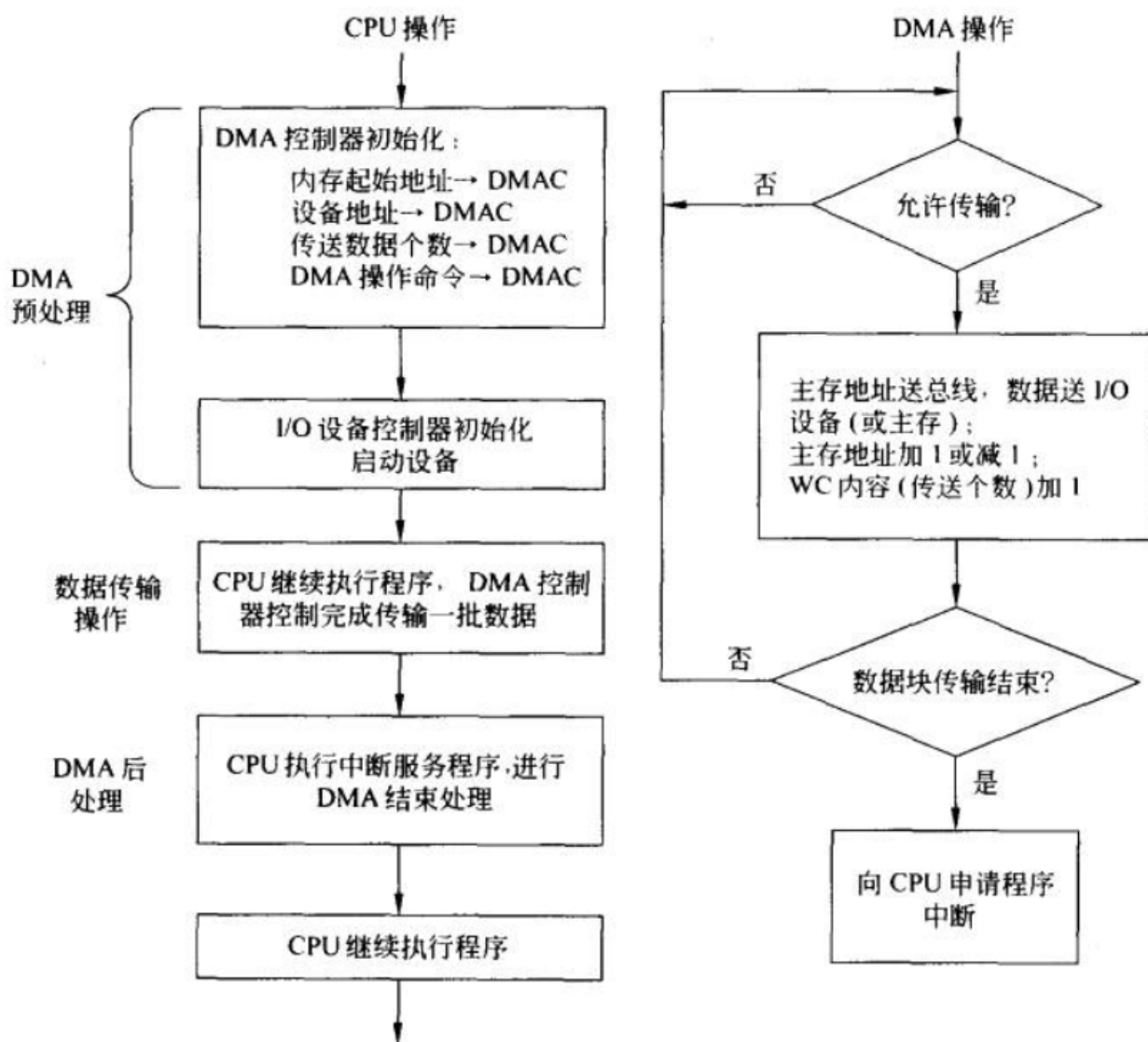


图 19 DMA 数据传送过程

预处理阶段：对 DMA 控制器进行一些初始设置，比如：内存首地址、数据长度、数据传送方式等。

数据传送阶段：DMA 控制器接管总线后进行的真正数据传输。

后处理阶段:数据传输结束后,DMA 控制器通过中断机制通知 CPU,然后 CPU 通过中断做该做的事。

6.5 总线

计算机系统中不同部件之间需要通过信号线相互连接起来,在彼此之间传送各种信息。

6.5.1 总线

各部件之间传送信息的公共通路称为**总线**。实质上总线是一个共享的传输媒介。当多个设备连接到总线上,其中任何一个通过总线传输的信号都能被连接到总线上的其它设备所接收。如果两个设备同时向总线发送信号,总线上的信号就会叠加而发生混乱,因此要对各功能部件使用总线的方式进行一定的限制,保证任何时候只有一个设备向总线发送信号。

1 总线的分类

对于总线的分类,由于所处的角度不同,有多种分类方法。

按总线所承担的任务可分为**内部总线**和**外部总线**。内部总线用于实现主机系统内部各功能模块(部件、板卡)之间的互联,外部总线用于实现主机系统与外部设备或其他主机系统之间的互联。其中,专门用于主机系统与外设之间互连的总线称为**设备总线**。

按总线所处的物理位置可分为**片内总线**、**板内总线**、**板间总线**和**外部总线**。片内总线实现芯片内部功能部件之间的连接,例如微处理器内部使用的总线,板内总线实现该电路板上各个集成电路芯片之间的互联,而板间总线则用于把各个功能模块(如 CPU 主存储器、I/O 接口适

配器等)连接到一起,构成主机系统,所以也称它为**系统总线**。

按总线所传送的信息类型可分为**地址总线**、**数据总线**和**控制总线**等。

按总线一次传送数据的位数又分为**串行总线**和**并行总线**。

按总线操作的定时方式,又有**同步总线**和**异步总线**之分。

2 总线标准

对某个具体总线而言,在围绕该总线进行设计、生产和使用时,都必须遵守该总线所定义的标准或规范。总线的规范主要从以下几个方面来描述总线的功能和特性。

(1) **逻辑规范**: 引脚信号的功能描述,包括信号的含义、信号的传送方向(发送接收或双向)、有效信号所采用的电平极性(高电平/低电平,正脉冲/负脉冲)及是否具有三态能力等。

(2) **时序规范**: 描述各信号有效/无效的发生时间以及不同信号之间相互配合的时间关系。例如当地址信号有效后,至少需要多长时间的延迟才能使读/写信号有效。

(3) **电器规范**: 总线上各个信号所采用的电平标准和负载能力。负载能力定义了总线理论上最多可以连接模块的数量。

(4) **机械规范**: 它定义了总线包括插槽/插头或插板的结构、形状、大小方面的物理尺寸、接插件机械强度;总线信号的布局、引脚信号的长度、宽度以及间距等。

(5) **通信协议**: 定义数据通过总线传输时采用的连接方法数据格式、发送速度等方面的规定。对串行总线而言会有这方面的规范。通信

协议通常还要分为若干层次。

3 总线的性能

总线的性能由多方面的因素决定，一个总线的性能水平主要由以下几个因素决定。

(1) 总线带宽：总线带宽表示在单位时间内，总线所能传输的最大数据量，一般用兆字节/秒 (MB/S) 来表示。

(2) 总线宽度：笼统地说，一个总线所设置的通信线路 (或线缆) 的数目称为该总线的宽度。具体来说，在一个总线内设置的用于传送数据的信号线的数目，称为数据总线宽度。同样也存在一个地址总线的宽度。

数据总线的宽度决定了一次可以同时传送的二进制信息的位数。在总线工作频率一定的条件下，数据总线单位时间内的数据传输量与数据总线的宽度成正比关系，因此数据总线的宽度是决定计算机系统性能的一个关键特性。

地址总线的宽度决定计算机系统的寻址能力。

(3) 总线的时钟频率：对于同步总线来说，由于采用统一的时钟脉冲作为定时基准，因此总线的时钟频率越高，总线上的操作就越快。显然，在数据总线宽度相同的情况下，较高的总线时钟频率会带来较大的数据吞吐量。

(4) 总线的负载能力：限定在总线上可以连接模块的最大数目。一般来说，都希望总线具有较高的带宽和较强的负载能力。

6.5.2 总线的控制与裁决

由于总线属于共享资源，为确保各个功能模块正确使用总线，需要有一个专门的部件来负责对总线的使用权进行管理和分配，这个专门部件就是**总线控制器**，也叫**总线仲裁器**。

由此一来，通过总线相互连接的各个功能模块在需要利用总线传输数据时，需要经历四个阶段：申请、仲裁、传输、结束

申请阶段：当某个模块需要利用总线和别的模块传输数据时，首先向总线控制器提出请求。

仲裁阶段：总线控制器接收到各个模块的请求后，根据一定的优先级规则进行裁决，确定谁将获得总线的使用权并且对获得使用权的模块作出回应。

传输阶段：获得使用权的模块接管总线并利用总线传输自己的数据。

结束阶段：获得总线使用权的模块利用总线传输完数据后，释放总线。

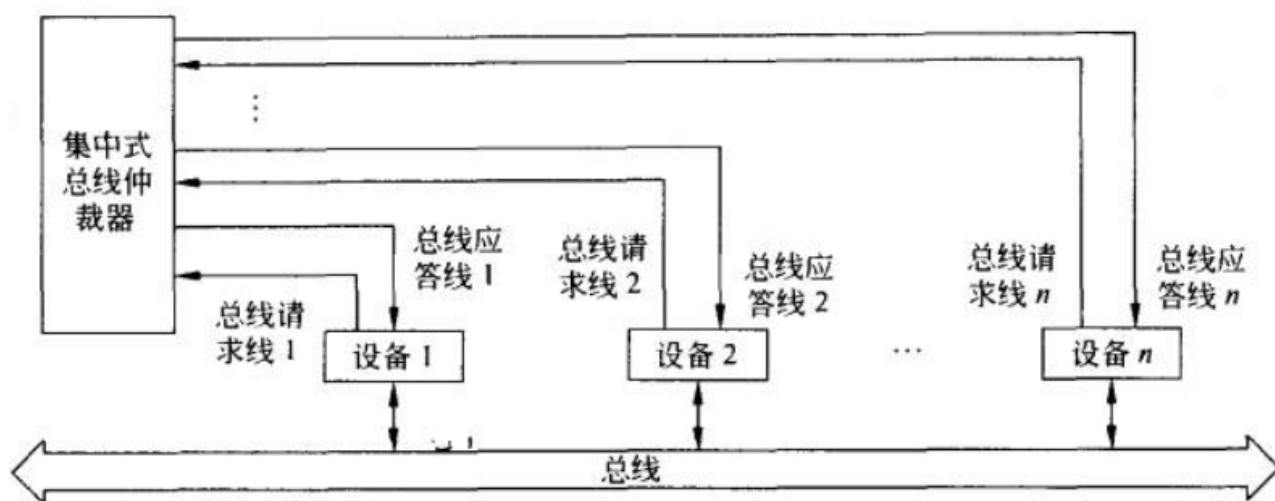


图 20 集中式并行总线仲裁

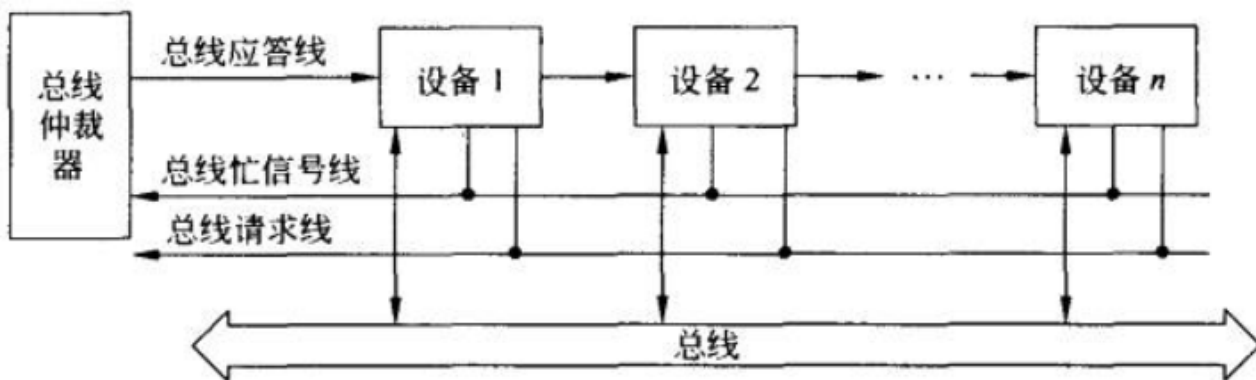


图 21 集中式串行总线仲裁

总线仲裁的方式可以采用集中式，也可以采用分布式。

集中式仲裁由单一总线仲裁器负责管理各个模块的总线请求并作出裁决，又可以细分为并行裁决和串行裁决，分别如图 20、图 21 所示。

6.5.3 计算机系统的总线结构

计算机系统总线的组织方法很多，按照总线的组织结构不同，可以在大体上分成单总线结构的计算机系统、双总线结构的计算机系统和多总线结构的计算机系统。

1 单总线结构

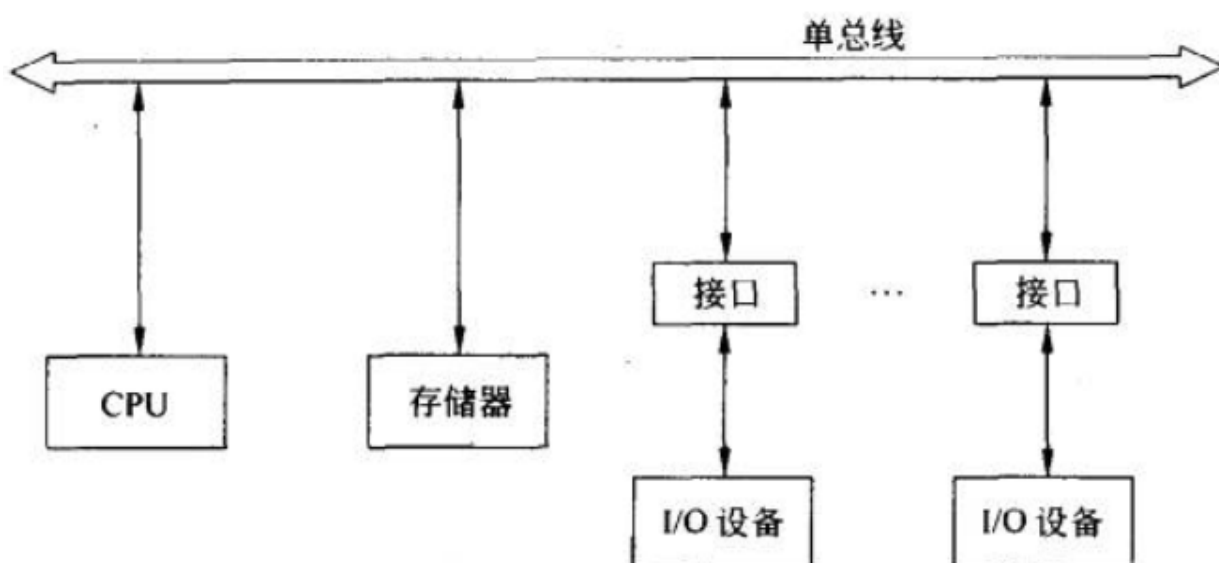


图 22 单总线结构

在单总线结构的计算机系统中，采用称为系统总线的一组总线来连接整个计算机系统中的各个功能部件，例如处理机模块、主存储器模块、I/O 设备控制器模块等，计算机系统中的所有外部设备通过设备控制器也挂在这条总线上。

在这种结构的计算机系统内部，各个功能部件之间的所有信息传送都通过系统总线来实现，如图 22 所示。

2 双总线结构

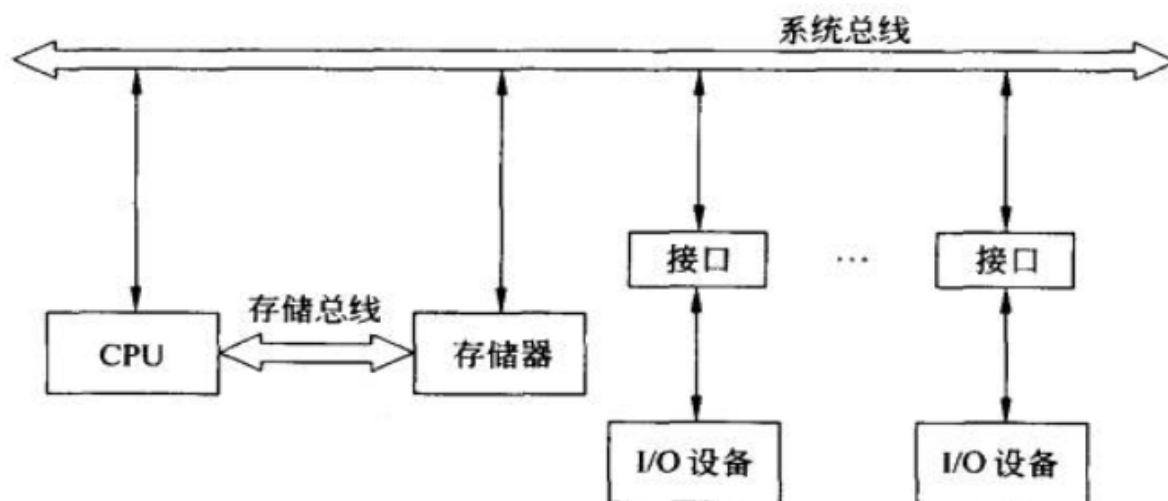


图 23 面向存储器的双总线结构

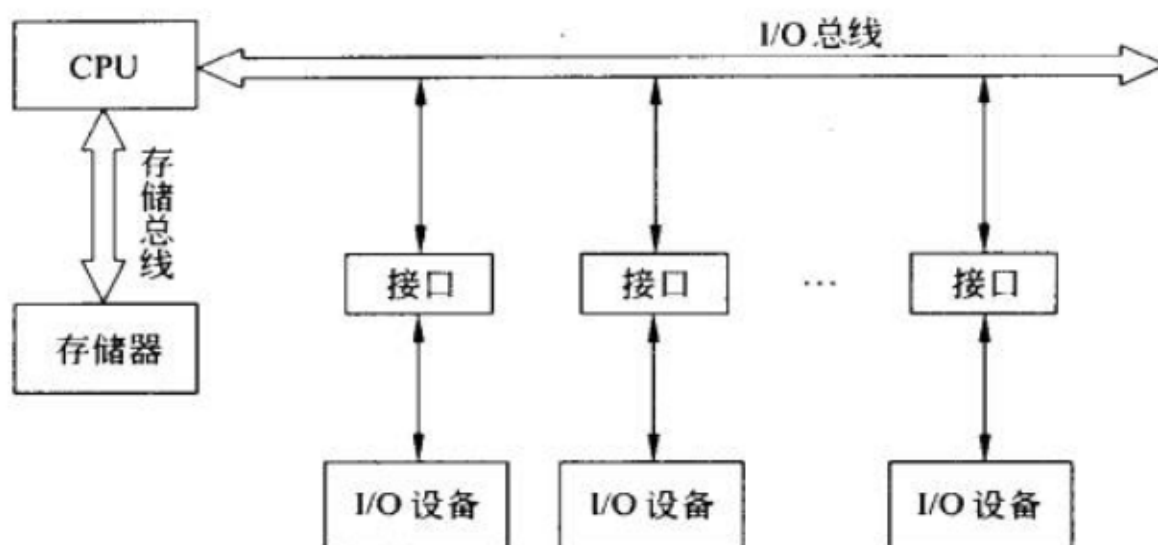


图 24 以 CPU 为中线的双总线结构

与单总线结构相比，在双总线结构中，增加了一条内存总线，CPU 访问主存单元时通过内存总线来实现，原有的系统总线则用来实现 CPU 与外设以及内存与外设之间的数据通信，如图 23、图 24 所示。

3 多总线结构

多总线结构的计算机系统是在双总线结构基础上增加 I/O 总线实现的一种计算机系统结构，目的是进一步提高计算机系统的工作效率。

这种总线结构是在计算机系统的各部件之间采用多条各自独立的总线来构成分层次的信息通路，这也是如今大、中型计算机系统的典型结构，如图 25 所示。

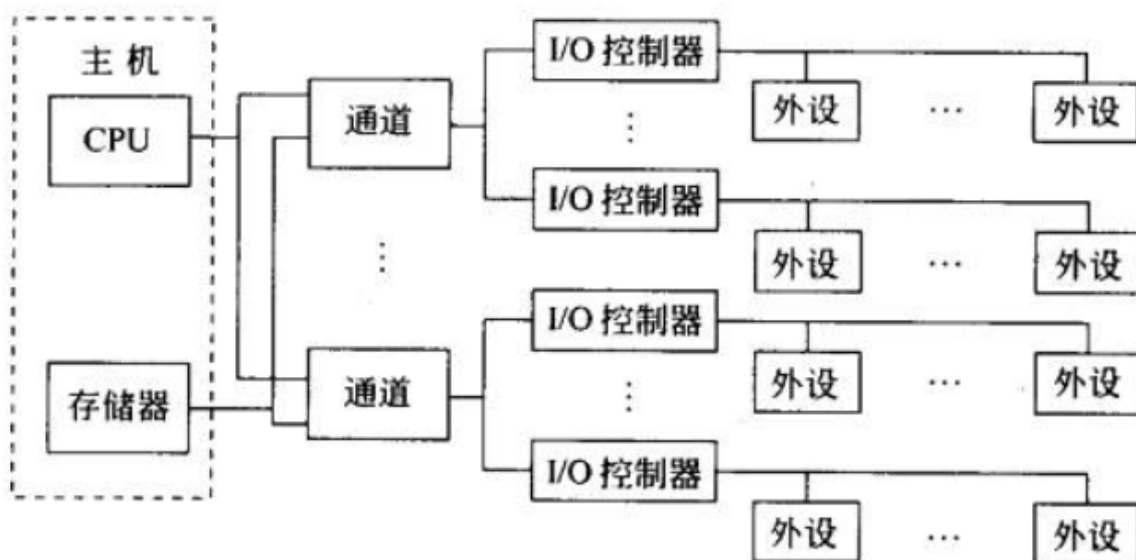


图 25 多总线结构

4 多层次总线结构

现代计算机系统往往会根据系统功能模块性能上的要求，设置不同层次、不同种类的总线，不会完全拘泥于上述三种总线结构，如图 26 所示。

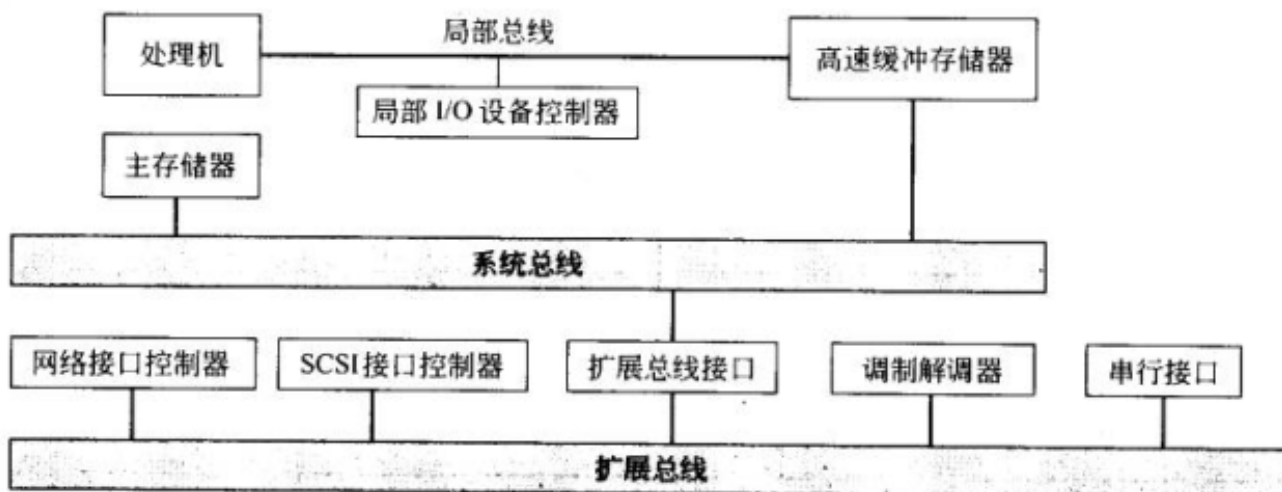


图 26 层次型多总线结构

6.5.4 常用总线

随着计算机技术及其应用的发展，总线技术也在不断发展之中，新型总线不断出现。

下面简单介绍几种总线。

1 内部总线

内部总线主要用于连接计算机主机系统内部的主要功能部件，一般位于机箱内部，所以有时也称系统总线。内部总线的标准和类型随着计算机系统的发展而不断发展，种类繁多，复杂性也在提高。

(1) 工业标准总线（ISA 总线）

工业标准总线（Industry Standard Architecture）是当年 IBM 公司为其生产的个人计算机（PC）所制定的总线标准。

ISA 总线，它是由 8 位的 PC 总线发展而来的 16 位总线。因此，ISA 总线又由两部分构成，即 ISA-8 总线和 ISA-16 总线。

ISA-8 总线也称 8 位 PC 总线，是在以 Intel 8088 为 CPU 的 IBM PC/XT 微机系统中使用的系统总线。

ISA-16 总线也称 PC/AT 总线，是在以 Intel 80286 为 CPU 的 IBM PC/AT 微机系统中使用的系统总线。

(2) 扩充的工业标准总线 (EISA 总线)

扩充的工业标准总线 (Extended Industry Standard Architecture) 是 EISA 集团专为 32 位 CPU 设计的系统总线。

(3) PCI 局部总线

20 世纪 90 年代后，随着图形处理技术和多媒体技术的广泛应用，特别是在以 Windows 为代表的图形用户界面应用得到普及之后，要求计算机系统具有对图形/图像数据的高速处理以及对显示数据的快速传输能力，这对总线技术提出了前所未有的挑战。原有的各类总线已远远不能满足应用的需求，总线成为整个计算机系统性能提升的主要瓶颈。

在此背景下，英特尔公司于 1991 年首先提出了 PCI (外围组件互连) 总线的概念，并联合 Compaq、AST、HP 等 100 多家公司成立了 PCI 集团兴趣小组，负责起草、制定 PCI 局部总线标准。

(4) PCI Express 总线

PCI Express 总线并不是 PCI 技术的延续，即 PCI Express 总线不是原有 PCI 总线的升级，而是一个全新的第三代总线。

2 设备总线

外部总线主要用于计算机系统的主机与外部设备之间的互联，所以有时也称之为设备总线。由于外部设备之间的差别很大，因此外部总线在形式上与系统总线也有很大的差别。如前所述，系统总线多半存在

于机箱内部，采用插槽形式集成在主电路板上，用户可以在扩充计算机系统功能时插入各种 I/O 接口适配器，如网卡、声卡等。而外部总线多在机箱外部，而且在机箱后部居多，其外形多样，差别很大，互不兼容。

(1) 小型计算机系统接口 (SCSI 接口)

小型计算机系统接口 (Small Computer System Interface, SCSI) 是用于小型、微型计算机和外围设备连接的一种接口标准，它可以支持包括磁盘驱动器、磁带机、光盘驱动器以及扫描仪在内的多种外部设备。虽然称它为接口，但 SCSI 接口实际上是一种外部并行总线。

(2) ATA 接口

ATA 接口 (AT 附件) 是微型计算机主板与硬盘等外部存储器之间的一种接口或总线。ATA 接口的前身是 IDE 接口，即集成设备电子部件。

(3) USB 接口

USB 是通用串行总线的简称，是由英特尔、康柏、微软、NEC、北方电信等多家世界著名的计算机和通信公司联合开发的一种新型串行接口总线标准。